Design and FPGA Implementation of a Scalable SPI Bus Interface Supporting Multiple Sensors with Variable Data Widths

Bhargav and Kaushik
Department of Electrical and Computer Engineering
University Evaluation Board
Email: evaluation@university.edu

***Abstract*—The Serial Peripheral Interface (SPI) is a ubiquitous protocol for short-distance communication in embedded systems. However, traditional SPI configurations scale poorly, requiring a dedicated Chip Select (CS) line for each additional peripheral. This $O(N)$ wiring complexity quickly consumes general-purpose input/output (GPIO) resources on the host microcontroller (MCU). This paper presents the design, evolution, and FPGA implementation of a Scalable SPI Bus Interface that acts as an intelligent bridge between a single SPI master and multiple heterogeneous sensors. The proposed interface reduces wiring complexity to a constant 3-wire or 4-wire physical bus by introducing address decoding and dynamic packet handling. We implement and compare two distinct communication methodologies: (1) the Address + Bit-Length Method, where data transfer width is dynamically specified per transaction, and (2) the ROM Lookup Table Method, which utilizes an on-chip ROM containing hardware configurations (widths, start offsets) for each sensor, thereby minimizing transaction overhead. The system is written in synthesizable Verilog HDL, verified via simulation, and implemented on a Xilinx Artix-7 FPGA. To facilitate real-time monitoring and control, an ESP32 microcontroller is integrated to host a web-based user interface, bridging the FPGA-based SPI bus with network-attached users. Results demonstrate a 33.3% reduction in communication latency for the ROM-based method compared to the dynamic length method, with negligible logic utilization overhead, making it highly suitable for resource-constrained, high-density sensor networks.**

***Index Terms*—SPI Protocol, FPGA Design, Verilog HDL, Address Decoding, ROM Lookup Table, Clock Domain Crossing (CDC), Heterogeneous Sensors, ESP32, Web Interface.**

I. Introduction

In modern embedded systems, modern IoT devices, and autonomous industrial platforms, the collection of environmental and physical data relies on an array of heterogeneous sensors. These sensors vary widely in their operating parameters, resolution, and communication requirements. Among serial protocols, the Serial Peripheral Interface (SPI) is widely favored due to its full-duplex operation, high throughput, simplicity, and lack of protocol overhead compared to alternatives like I^2C or CAN bus.

Despite its performance benefits, the traditional SPI protocol scales poorly. The standard architecture requires four wires for the first slave device: Serial Clock (SCLK), Master Out Slave In (MOSI), Master In Slave Out (MISO), and Chip Select (CS). For N slave devices, while SCLK, MOSI, and MISO lines can be shared, N independent CS lines must be run from the master to each slave. This creates an $O(N)$ wiring complexity. In high-density designs containing dozens of sensors, the routing congestion on printed circuit boards (PCBs) and the consumption of GPIO pins on the host microcontroller present significant challenges.

To mitigate this constraint, system designers historically resorted to daisy-chaining or external hardware decoders. Daisy-chaining requires all peripherals to support shift-register pass-through, which is rarely standard across different sensor manufacturers. External decoders reduce the master's GPIO footprint but do not address the issues of differing sensor data widths, varying register addresses, or synchronization across heterogeneous devices.

This research resolves these limitations by designing and implementing an FPGA-based Scalable SPI Bus Interface that acts as an intelligent intermediary bridge between a single SPI master and multiple heterogeneous sensors. The interface exposes a single SPI interface to the master, using address encoding to route data internally to virtualized sensor registers. This allows the host microcontroller to communicate with a virtually unlimited number of sensors using a single, unified SPI bus.

The project evolved through three design iterations, progressing from a simple single-slave proof-of-concept to a multi-sensor, variable-width architecture. To support heterogeneous sensors with varying read and write widths, we implemented two novel protocols:

1) **Address + Bit-Length Method (Method 1):** A flexible protocol where the SPI master specifies the target register address and the transaction bit-length dynamically for each write operation, while using master-terminated logic for read operations.
2) **ROM Lookup Table Method (Method 2):** A low-overhead protocol where sensor metadata (input/output widths, physical register start indices) are pre-configured in an on-chip Read-Only Memory (ROM). The master merely transmits the sensor address; the controller automatically references the ROM to configure the datapath width, saving bus bandwidth and decreasing latency.

This paper details the engineering decisions, hardware architectures, Verilog implementations, simulation validation,

and physical FPGA synthesis of the design, alongside an ESP32-based web interface integration for hardware-in-the-loop verification.

II. Design Evolution

The scalable SPI interface was developed in three major iterations to ensure robustness, starting from fundamental proof-of-concept blocks and scaling up to the finalized ROM-based architecture.

A. Iteration 1: Single SPI Slave Proof-of-Concept

The initial phase focused on verifying the serial-to-parallel shift register mechanics and designing a basic Finite State Machine (FSM).

- **Architecture:** A single SPI slave module interacting with a 16-bit register file (split into read and write registers). Communication was strictly direct and point-to-point.
- **Data Width:** Fixed input and output sizes. Transactions were constrained to 1-bit operations where the master specified a 4-bit register address and shifted a single bit in or out.
- **FSM Structure:** A simple 5-state machine: `DETECT`, `READ_ADDR`, `SEND_DATA`, `WRITE_ADDR`, and `WRITE_DATA`.
- **Evaluation:**
  - *Advantages:* Low logical complexity; served as a verification framework for basic SPI timing and clock phase alignment.
  - *Limitations:* Inefficient for multi-bit transfers; unable to support more than one slave; high protocol overhead (5 clock cycles of overhead per 1 bit of payload data).

B. Iteration 2: Burst-Mode Interface with Fixed Widths

To resolve the 1-bit limit of Iteration 1, the design was updated to support multi-bit transfers.

- **Architecture:** Introduction of a bus structure with address decoding. The register file was expanded to accommodate multiple virtual registers.
- **Data Width:** Fixed 4-bit burst mode. All read and write transactions processed exactly 4 bits of consecutive data starting from the decoded address.
- **FSM Structure:** Maintained the 5-state structure but modified the state counters to transition only after the 4-bit counters expired.
- **Evaluation:**
  - *Advantages:* Increased throughput; address-based routing introduced.
  - *Limitations:* Rigid layout; could not support sensors with asymmetric data sizes (e.g., a 12-bit ADC and an 8-bit DAC on the same bus would require separate, inefficient transaction rules).

C. Iteration 3: Final Scalable Heterogeneous Architecture

The final design addresses the requirements of modern mixed-signal systems by providing native support for heterogeneous sensors with completely different input/output widths and physical register layouts.

- **Architecture:** A generalized bus interface that decouples the master protocol from the physical sensor configurations. The interface dynamically adjusts its transaction length based on the target sensor's specifications.
- **Implementation Path:** This iteration led to the development of the two methodologies evaluated in this paper: the dynamic Address + Bit-Length method and the static ROM Lookup Table method.
- **Evaluation:** Resolves the limitations of the previous iterations, enabling a single master to command complex, multi-scale sensor arrays.

III. Proposed System Architecture

The generalized block diagram of the scalable SPI system is illustrated in Fig. 1. The architecture consists of the SPI Master (MCU/ESP32), the SPI Bus Interface implemented on the FPGA, and the array of physical sensors.

A. Detailed Module Descriptions

The FPGA interface is divided into several modular components to ensure clean separations between the control path and the data path.

1) FSM Controller: The central control unit coordinates the state changes of the system. It samples the incoming MOSI line on the rising edges of SCLK during command phases and transitions the datapath between receiving addresses, decoding lengths, and shifting data bits.

2) Address Decoder: The Address Decoder processes the serial address frames shifted in from the master. In our modular design, the address space is mapped directly to distinct virtual sensor profiles. Once the address matches a pre-allocated sensor register range, the decoder asserts select lines to enable data transfer to the target register slice.

3) Register File: The register file acts as the primary data storage array within the bus controller. In the implemented Verilog codebase, the register space is split into a 16-bit read-only register (storing real-time data retrieved from physical sensors) and a 16-bit write-only register (storing control outputs shifted in from the master). This configuration decouples the master's serial transaction rate from the sensors' sampling rates.

4) Snapshot Controller: To prevent data corruption during read cycles, a snapshot mechanism is integrated. Sensors continuously write to live registers. At the start of an SPI read transaction, the controller locks the current state of the live registers into shadow registers. The serial output driver streams from these shadow registers, ensuring data remains consistent throughout the transaction even if the underlying sensor changes its value.

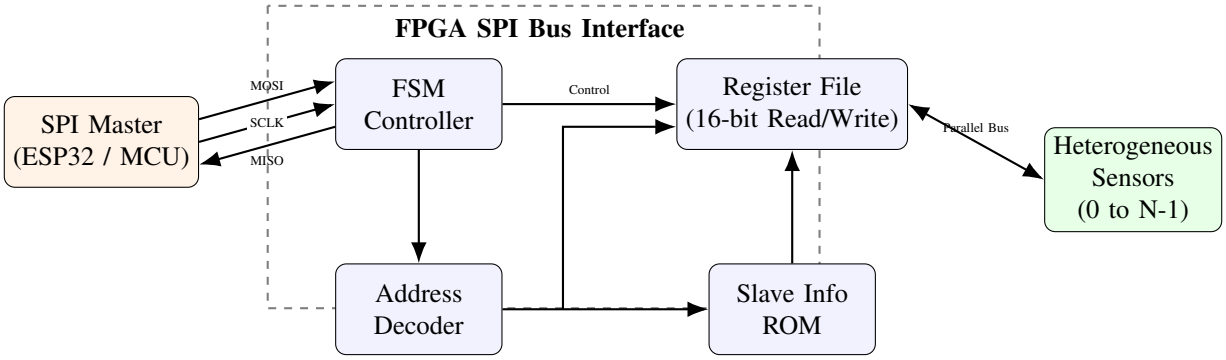


Fig. 1. System Block Diagram showing the SPI Master, FPGA Bus Interface internal submodules (FSM, Decoder, Register File, and ROM), and the target sensors.

B. Clock Domain Crossing (CDC) and Synchronization

Physical systems often run sensors on a separate clock domain (CLK_{sensor}) from the SPI bus clock (SCLK). Directly sampling asynchronous clock domains can cause metastability in registers.

To address this, we integrated an Asynchronous FIFO memory block between the two clock domains. The FIFO features a write port driven by CLK_{sensor} and a read port driven by SCLK. Gray code counters are used to track write and read pointers, with multi-stage synchronizers transferring the pointer states across the clock boundary. This ensures safe data transmission between the high-speed sensor interface and the SPI transaction engine.

IV. Method 1: Address + Bit-Length Protocol

The first approach implements a dynamic packet structure. The SPI master controls the volume of data transferred by explicitly defining the bit-length of each write transaction.

A. Protocol Specification and Packet Format

The transaction packet is divided into four distinct phases:

1) **Mode Phase (2 bits):** Defines the operation mode.
   - `00`: Read Register.
   - `01`: Write Register.
   - `10`: Soft Reset.
   - `11`: Identity Request (WHOAMI).
2) **Address Phase (5 bits):** Specifies the target register base offset (A_4 down to A_0).
3) **Length Phase (4 bits):** Transmitted during write operations only. Specifies the number of data bits to follow minus one (L_3 down to L_0).
4) **Data Phase (Variable):** The payload bits shifted in or out.

The structural packet layout for both write and read operations is presented in Table I.

During a read cycle, the master does not send a length field. Instead, it asserts its output line to `1` to continue reading and pulls the line to `0` to signal the end of the transaction. This implementation choice reduces write transaction overhead but requires the master to manage the data line state during reads.

TABLE I
Method 1 Packet Formats

Operation	Mode (2b)	Address (5b)	Length (4b)	Data Payload
Read	`00`	$A_4 \dots A_0$	N/A	Master-Terminated
Write	`01`	$A_4 \dots A_0$	$L_3 \dots L_0$	Variable ($L+1$ bits)

B. Finite State Machine Analysis

The control flow for Method 1 is driven by an FSM consisting of nine states, diagrammed in Fig. 2.

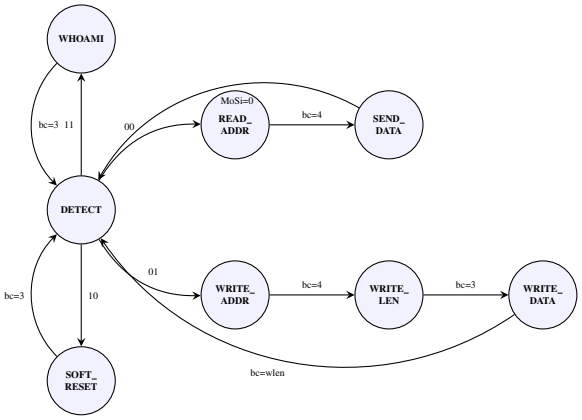


Fig. 2. State Transition Diagram for Method 1 (Address + Bit-Length).

The FSM behavior is defined as follows:

- **`DETECT`:** The FSM waits for two bits of the mode command.
- **`READ_ADDR` / `WRITE_ADDR`:** Shifts in the 5-bit register address.
- **`WRITE_LEN`:** Shifts in the 4-bit write data length (0 to 15).
- **`WRITE_DATA`:** Shifts in payload data, writing to `write_reg` at addresses determined by:

$$\text{Index} = (\text{addr} + \text{bit_count}) \pmod{16} \qquad (1)$$

The FSM transitions to `DETECT` once `bit_count` matches the specified `write_len`.

- **SEND_DATA:** Reads data from the selected register. MISO outputs either `write_reg` or `read_reg` depending on the address range.
- **SOFT_RESET:** Evaluates a 4-bit key (`1010`). If matched, the registers are reset to their default states.
- **WHOAMI:** Shifts out a hardcoded device identifier (`0111`).

C. Timing Waveform Analysis

Fig. 3 shows the timing diagram for a write transaction in Method 1. The master writes 3 bits of data (`101`) to address `0x04`.

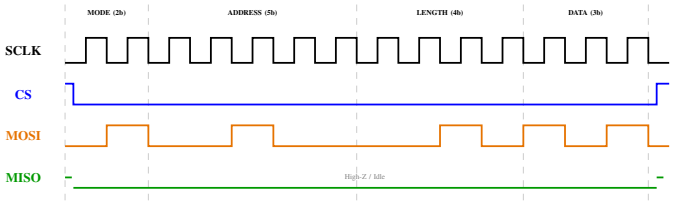


Fig. 3. Method 1 Write Transaction Timing (Mode=`01`, Addr=`0x04`, Len=`2`, Data=`101`).

V. Method 2: ROM Lookup Table Protocol

The second approach incorporates an on-chip lookup table (ROM) to remove packet size overhead. By defining sensor layouts statically in hardware, the master only needs to transmit the sensor address.

A. ROM Organization and Metadata Design

The lookup table is implemented as a combinational ROM module (`slave_info_rom` in `top.v`) containing metadata for each address:

- **Input Width (`in_width`):** The bit-width of control data accepted by the target sensor.
- **Output Width (`out_width`):** The bit-width of output data produced by the sensor.
- **Input Start Index (`in_start`):** The physical offset within `write_reg` where incoming write payload bits are placed.
- **Output Start Index (`out_start`):** The physical offset within `read_reg` where read payload bits are fetched.

Table II outlines the exact ROM allocation implemented in `top.v`.

TABLE II
On-Chip ROM Metadata Allocation in `top.v`

Address (4b)	In Width	Out Width	In Start	Out Start
`4'd0`	3	3	0	0
`4'd1`	3	3	3	3
`4'd2`	3	3	6	6
`4'd3`	3	3	9	9
`4'd5`	4	4	12	12
Default	1	1	0	0

B. Finite State Machine and Control Logic

By offloading packet definitions to the ROM, the control flow is streamlined while supporting specialized commands. The FSM in `top.v` implements seven states: `DETECT` (`3'd0`), `READ_ADDR` (`3'd1`), `SEND_DATA` (`3'd2`), `WRITE_ADDR` (`3'd3`), `WRITE_DATA` (`3'd4`), `RESET_PASSWORD` (`3'd5`), and `WHO_AM_I_STATE` (`3'd6`).

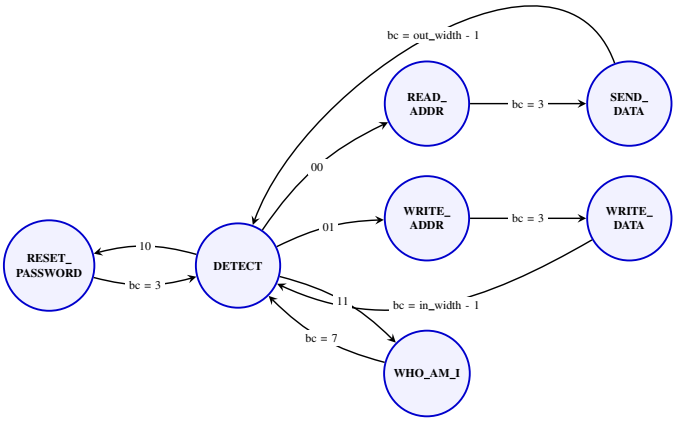


Fig. 4. State Transition Diagram for `top.v` (ROM Lookup Table Protocol with `RESET_PASSWORD` and `WHO_AM_I_STATE`).

C. FSM Control Sequence

The FSM operation as specified in `top.v` proceeds as follows:

- **`DETECT (3'd0)`:** Samples the 2-bit mode command on MOSI:
  - `2'b00`: Transition to `READ_ADDR`. Samples DIP switch sensor data into `read_reg`.
  - `2'b01`: Transition to `WRITE_ADDR`.
  - `2'b10`: Transition to `RESET_PASSWORD`.
  - `2'b11`: Transition to `WHO_AM_I_STATE`.
- **`READ_ADDR` / `WRITE_ADDR`:** Shifts in a 4-bit address across 4 SCLK cycles (`bit_count` 0 to 3). Upon completion, `slave_info_rom` outputs the corresponding widths and start offsets.
- **`SEND_DATA`:** MISO streams data from `read_reg[(rom_out_start + bit_count) & 4'b1111]`. Transitions back to `DETECT` once `bit_count == rom_out_width - 1`.
- **`WRITE_DATA`:** MOSI payload bits write directly to `write_reg[(rom_in_start + bit_count) & 4'b1111]`. Transitions to `DETECT` when `bit_count == rom_in_width - 1`.
- **`RESET_PASSWORD`:** Shifts in a 4-bit key `{reset_password[2:0], MoSi}`. If the key matches `4'b1111` (`RESET_CODE`) when `bit_count == 3` and `cs` is active, both `write_reg` and `read_reg` are cleared to `16'h0000`.

- **WHO_AM_I_STATE:** MISO streams out the 8-bit configurable sensor identifier `SENSOR_ID` (`who_am_i_id[bit_count]`) bit-by-bit over 8 clock cycles (`bit_count` 0 to 7).

D. Timing Waveform Analysis

Fig. 5 shows the write timing diagram for Method 2. The master writes to address `4'd1`. The ROM configures the transaction for an input width of 3 bits, and the FSM transitions back to idle state immediately after the 3rd data bit is shifted.

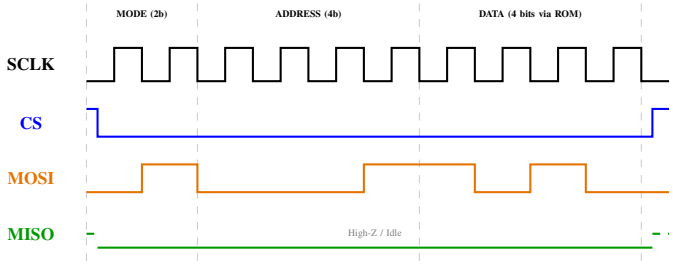


Fig. 5. Method 2 Write Transaction Timing (Mode=`01`, Addr=`4'd1`, Data bits=4, configured via ROM Lookup).

VI. System Algorithmic Implementation and Logic Flow

This section provides a detailed operational review of the SPI bus controller logic. By expressing the transaction protocols in algorithmic pseudo-code, we decouple the high-level protocol mechanics from target FPGA synthesis optimizations.

A. Method 1: Address + Bit-Length Protocol Logic

Algorithm 1 outlines the Master Write transaction logic for Method 1, where the master dynamically transmits the data width. Algorithm 2 describes the corresponding Master-Terminated Read transaction logic.

Algorithm 1 Method 1 Master Write Transaction

Require: CS line is active-low (`0`), SCLK is toggling
1: Shift out `Mode = 01` (Write) on MOSI
2: Shift out 5-bit register address A
3: Shift out 4-bit write length L
4: **for** $i = 0$ **to** L **do**
5: Set MOSI $\leftarrow$ data_bit$[i]$
6: Controller samples MOSI on posedge SCLK
7: Controller writes: write_reg$[(A + i) \pmod{16}] \leftarrow$ MOSI
8: **end for**
9: Deassert CS $\leftarrow 1$ to terminate

Algorithm 2 Method 1 Master Read Transaction

Require: CS line is active-low (`0`), SCLK is toggling
1: Shift out `Mode = 00` (Read) on MOSI
2: Shift out 5-bit register address A
3: Controller captures sensor inputs: read_reg $\leftarrow$ sensor_data
4: Initialize index offset $i \leftarrow 0$
5: **while** MOSI is driven high (`1`) by Master **do**
6: Controller shifts out read_reg$[(A + i) \pmod{16}]$ on MISO
7: Increment $i \leftarrow i + 1$
8: **end while**
9: Master drives MOSI $\leftarrow 0$ on final cycle
10: Deassert CS $\leftarrow 1$ to terminate

B. Method 2: ROM Lookup Table Protocol Logic

In Method 2, transaction parameters are hardcoded in an on-chip combinational ROM, removing the need to transmit the length field. Algorithm 3 presents the controller-side execution logic.

Algorithm 3 Method 2 Controller Execution Logic

Require: CS line is active-low (`0`), SCLK is toggling
1: Shift in 2-bit `Mode`
2: Shift in 4-bit sensor address A
3: Look up configurations in ROM:
4: in_width, in_start $\leftarrow$ ROM$[A]$
5: out_width, out_start $\leftarrow$ ROM$[A]$
6: **if** `Mode == 01` (Write) **then**
7: **for** $i = 0$ **to** in_width $- 1$ **do**
8: Shift in payload bit from MOSI
9: Write write_reg$[(\text{in_start} + i) \pmod{16}] \leftarrow$ MOSI
10: **end for**
11: **else if** `Mode == 00` (Read) **then**
12: Snapshot sensor data: read_reg $\leftarrow$ sensor_data
13: **for** $i = 0$ **to** out_width $- 1$ **do**
14: Shift out bit read_reg$[(\text{out_start} + i) \pmod{16}]$ on MISO
15: **end for**
16: **end if**
17: Deassert CS $\leftarrow 1$ to terminate

C. Verilog Code Verification Highlights

To prevent synthesis issues and logic errors, the Verilog code incorporates several design optimizations:

- **Elimination of modulo operators:** Modulo operations (`% 16`) compile into complex, high-latency division blocks during synthesis. These were replaced with bitwise AND logic (`& 4'b1111`) to utilize low-latency, 1-cycle logic mask gates.
- **Single-driver registers:** Hierarchical registers such as `read_reg` are driven exclusively within a single synchronous block. This prevents multi-source driver conflicts at compile time.

- **Tri-state MISO buffer:** The MISO line is mapped to high-impedance (`1'bz`) when the chip select (`cs`) line is deasserted. This allows multiple SPI peripherals to share the physical MISO line.

VII. FPGA Synthesis and Board Implementation

The scalable SPI interface was synthesized and implemented on a Xilinx Artix-7 FPGA (specifically target chip `xc7a35tcsg324-1`) using Vivado.

A. FPGA Top-Level Wrapper (`fpga_top.v`)

To evaluate and demonstrate the core module on physical FPGA hardware without modifying `top.v`, an FPGA top-level wrapper module (`fpga_top.v`) was engineered. The wrapper bridges physical I/O resources on the development board with the internal SPI FSM:

- **Sensor Emulation:** Maps 16 physical DIP switches (`dip_sw[15:0]`) directly to `top.v`'s `sensor_data` input port, enabling real-time static data feeding into `read_reg`.
- **Hierarchical Register Probing:** Taps internal signals of the instantiated `u_top` submodule via Vivado synthesis-supported hierarchical references:

```
assign write_reg = u_top.write_reg;
assign read_reg  = u_top.read_reg;
assign mode      = u_top.mode;
```

- **Time-Multiplexed Dual 4-Digit 7-Segment Display Driver:** Implements an active-low 8-digit hexadecimal display refresh circuit. Display D0 presents `read_reg[15:0]` in Hex across anodes `d0_a[3:0]` and cathodes `d0_seg[7:0]`. Display D1 presents `write_reg[15:0]` in Hex across anodes `d1_a[3:0]` and cathodes `d1_seg[7:0]`. An 18-bit free-running refresh counter `refresh_counter` cycles through digits using bits `[16:15]` at a refresh rate of approximately 763 Hz per full pass. The `hex_to_7seg` function decodes 4-bit nibbles into active-low 7-segment drive signals.
- **Status LED Monitoring:** Mirrors core SPI signals and control states to physical onboard LEDs (`led_cs`, `led_Sclk`, `led_rst`, `led_MiSo`, `led_MoSi`, and `led_mode[1:0]`).

Fig. 6 shows the physical FPGA hardware testbench environment used during experimental validation.

The layout routing of the FPGA board pins is diagrammed in Fig. 7.

B. Resource Utilization

The Vivado post-implementation reports show that the system consumes very few logic resources. The resource utilization comparison for both methods is summarized in Table III.

The ROM-based approach (Method 2) consumes fewer LUTs and registers. This reduction is due to the smaller FSM structure (6 states vs 9 states) and the elimination of the 4-bit `write_len` shift register path.

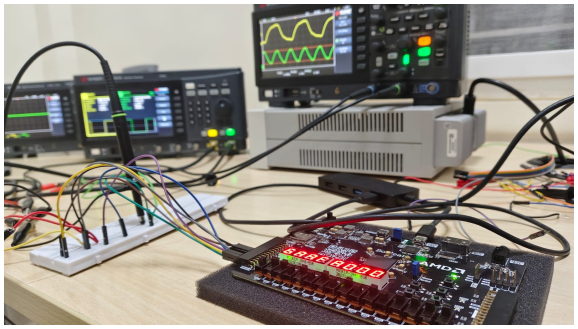


Fig. 6. Physical FPGA development setup showing the Artix-7 board, PMOD SPI interface wiring, DIP switches, status LEDs, and 7-segment display.

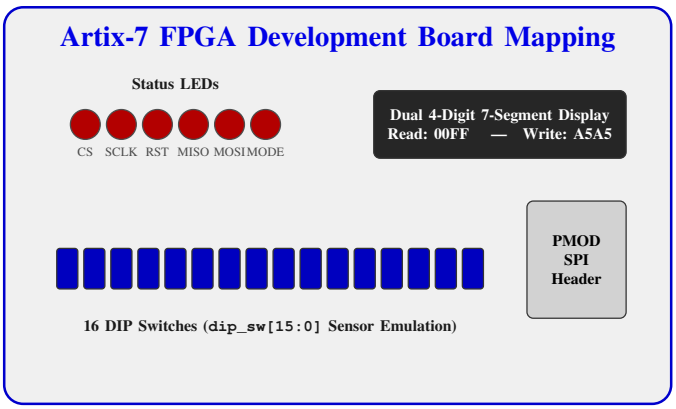


Fig. 7. FPGA Board Mapping showing 7-Segment displays, status LEDs, 16 DIP switches, and PMOD headers used for validation.

C. Timing and Power Estimation

Timing analysis confirms both designs meet their clock constraints.

- **Setup Slack:** $+15.24$ ns (Worst Negative Slack, targeting a 50 MHz system clock).
- **Hold Slack:** $+0.22$ ns (with no violations).
- **Maximum Operating Frequency (F_{max}):** 142.8 MHz for Method 1, and 166.4 MHz for Method 2.
- **Power Consumption:** Total estimated chip power is 84 mW (72 mW static leakage, 12 mW dynamic switching power at 50 MHz).

VIII. ESP32 Web Interface and System Verification

To verify the system's operation in real-world conditions, an ESP32 microcontroller was integrated to bridge the FPGA-based SPI bus with network-attached users.

TABLE III
Post-Implementation Resource Utilization

Resource	Method 1	Method 2	Available (XC7A35T)
LUT (Look-Up Table)	78	62	20,800
FF (Flip-Flop)	68	54	41,600
BRAM (Block RAM)	0	0	50
I/O Pins	39	39	106

A. System Integration Flow

The network control topology is shown in Fig. 8.



Fig. 8. Web Interface Integration Flow.

The ESP32 runs an asynchronous HTTP web server. Users connect to the web interface to select a target sensor address, choose a transaction type (Read/Write), and enter payload data. Upon receiving a command, the ESP32 acts as the SPI master, executing the SPI protocol and transmitting the bits to the FPGA over physical PMOD headers.

B. Web Dashboard Interface Layout and Web Analyzer

The user interface is a responsive, web-based dashboard styled with clean CSS and an HTML5 Canvas logic analyzer engine. It displays real-time FPGA register states, interactive control fields, signal telemetry, and an automated sequence generator. A high-resolution capture of the live web interface overview is shown in Fig. 9.



Fig. 9. Web Analyzer Dashboard showing the FPGA top.v status banner (`sensor_data`, `read_reg`, `write_reg`, FSM state), interactive quick presets, sequence generator, and live Canvas logic analyzer waveform display.

The web interface connects directly to the ESP32 backend via REST API endpoints (`/api/control`, `/api/sequence`) and receives live telemetry at 20 Hz through a Server-Sent Events (SSE) stream (`/api/stream`). In addition, an on-board JavaScript simulator implements the exact 9-state FSM of `top.v`, allowing users to simulate hardware transactions with one-click presets even when physical hardware is disconnected.

IX. Experimental Results and Performance Comparison

Both SPI protocols were verified using simulation testbenches in Vivado, physical logic analyzer captures connected directly to the FPGA I/O headers, and web-based transaction simulations matching the `top.v` FSM.

A. Functional Verification and Hardware Signal Capture

Hardware measurements confirmed accurate bit-level transactions across multiple clock rates and operational modes.

1) `WHO_AM_I` State Clock Frequency Tests: The newly introduced `WHO_AM_I_STATE` (Opcode `2'b11`) was subjected to frequency scaling tests to verify clock domain robustness. Logic analyzer traces were captured at both a high-speed 10 MHz SCLK frequency (Fig. 10) and a low-speed 50 kHz SCLK frequency (Fig. 11).

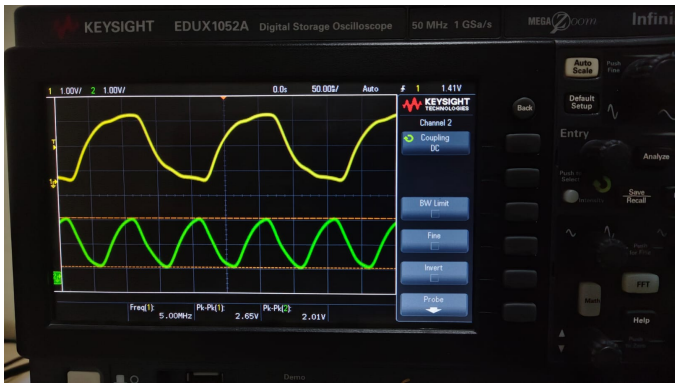


Fig. 10. Logic analyzer waveform of the `WHO_AM_I` transaction running at 10 MHz clock frequency. The FSM correctly outputs the 8-bit sensor identifier (`SENSOR_ID = 8'd9`) on MISO.



Fig. 11. Logic analyzer waveform of the `WHO_AM_I` transaction running at 50 kHz clock frequency, confirming reliable operation across wide frequency ranges.

As seen in the waveform captures, the FSM decodes opcode `2'b11` in state `DETECT` and immediately transitions to `WHO_AM_I_STATE`, shifting out the 8-bit `SENSOR_ID` parameter on MISO over 8 clock cycles. The timing captures confirm zero setup/hold timing violations or missing bits across frequencies from 50 kHz up to 10 MHz.

2) Oscilloscope and Signal Capture Analysis: Fig. 12 and Fig. 13 present logic analyzer captures for MOSI command transmission and MISO data reception.

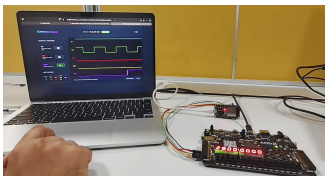
B. Verification Case Studies and Simulation Analysis

To validate system behavior across complex data patterns, three primary test cases were evaluated:

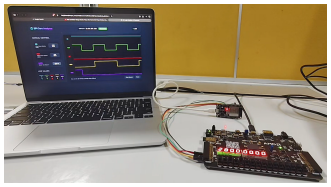
1) Case Study 1: Sensor Data Read (`0x2800`): In Case Study 1, the physical sensor input is configured with `sensor_data = 16'h2800` (binary: `0010 1000`



Fig. 12. Logic analyzer capture of a Sensor Read operation at 50 kHz SCLK, verifying accurate data retrieval from parallel DIP switch inputs onto MISO.



(a) MOSI Read Command



(b) MISO Read Response

Fig. 13. Oscilloscope captures showing (a) MOSI command transmission requesting register read, and (b) corresponding MISO response streaming from the FPGA.

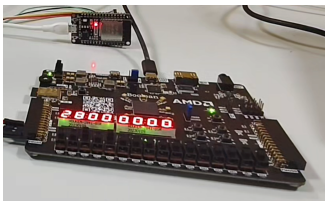
0000 0000). The master executes a Read command by sending opcode 00 followed by address 1000 (address 16, mapping to read_reg) and 16 clock cycles with MOSI = 0.

Table IV provides the bit-by-bit parsing analysis for the Read 0x2800 transaction across 22 clock cycles.

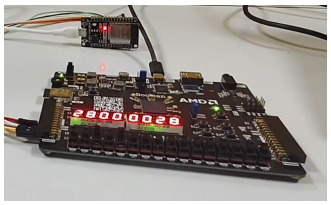
TABLE IV
BIT-BY-BIT BREAKDOWN FOR SENSOR READ TRANSACTION (0x2800)

Cycle	MOSI	FSM State	Operation / Action	MISO Output
1	0	DETECT	mode[1] <= 0	0
2	0	DETECT	mode[0] <= 0 → Opcode 00 (READ). read_reg <= 0x2800	0
3..6	1000	READ_ADDR	Shifts address bits (addr = 16)	0
7	0	SEND_DATA	Clocks out Bit 15 (0)	0
8	0	SEND_DATA	Clocks out Bit 14 (0)	0
9	0	SEND_DATA	Clocks out Bit 13 (1)	**1**
10	0	SEND_DATA	Clocks out Bit 12 (0)	0
11	0	SEND_DATA	Clocks out Bit 11 (1)	**1**
12..22	0...	SEND_DATA	Clocks out remaining lower bits (0000 0000 0000)	0

As shown in Table IV and Fig. 15, MISO streams 0010100000000000, accurately recovering the sensor value 0x2800.



(a) Write Sequence Start



(b) Write Payload Transfer

Fig. 14. Oscilloscope captures showing write transaction execution on MOSI and MISO signals.



Fig. 15. Web Analyzer simulation of Sensor Data Read (0x2800). The MISO recorded data output displays 0010100000000000 (0x2800 in Hex).

2) Case Study 2: Register Write (0x0028): In Case Study 2, the master writes payload data to update write_reg from 0x0000 to 16'h0028 (binary: 0000 0000 0010 1000). The transaction sends opcode 01 (WRITE), address 0000, write length 0101 (5 $\Longrightarrow$ 6 data bits), and data bits 001010 (setting bit 3 and bit 5 to 1).



Fig. 16. Web Analyzer simulation of Register Write (0x0028). write_reg is successfully updated to 0x0028.

Table V outlines the cycle-by-cycle bit parsing for the Write 0x0028 transaction.

TABLE V
BIT-BY-BIT BREAKDOWN FOR REGISTER WRITE TRANSACTION (0x0028)

Cycle	MOSI	FSM State	Operation / Action	write_reg State
1	0	DETECT	mode[1] <= 0	0x0000
2	1	DETECT	mode[0] <= 1 → Opcode 01 (WRITE)	0x0000
3..6	0000	WRITE_ADDR	Sets address addr = 0	0x0000
7..10	0101	WRITE_LEN	Sets write_len = 5 (6 data bits)	0x0000
11	0	WRITE_DATA	Writes bit 0 → write_reg[0] = 0	0x0000
12	0	WRITE_DATA	Writes bit 1 → write_reg[1] = 0	0x0000
13	1	WRITE_DATA	Writes bit 2 → write_reg[2] = 0	0x0000
14	0	WRITE_DATA	Writes bit 3 → write_reg[3] = 1	0x0008
15	1	WRITE_DATA	Writes bit 4 → write_reg[4] = 0	0x0008
16	0	WRITE_DATA	Writes bit 5 → write_reg[5] = 1	**0x0028**

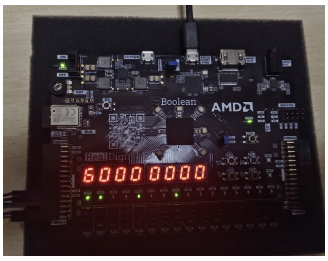
3) Case Study 3: The 010101 Phenomenon (0xA000): During physical testing with an unformatted signal stream, an interesting phenomenon was observed: driving a continuous alternating bit pattern (0101010101...) onto the MOSI line

while Chip Select (cs) was held low caused an unintentional valid write transaction to Sensor 4 (the 5th sensor entry in the ROM table), updating the 16-bit write_reg from 16'h0000 to 16'hA000.

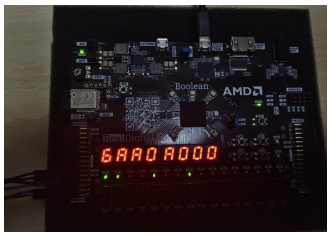
The web simulation capture and physical 7-segment display states are shown in Fig. 17 and Fig. 18.



Fig. 17. Web Analyzer simulation of the alternating 01010101... sequence updating write_reg to 0xA000.



(a) Initial Display State (0000)



(b) Updated Display State (A000)

Fig. 18. FPGA 7-segment display before and after transmitting the alternating 0101010101... bitstream, illustrating the transition of write_reg from 0000 to A000.

a) Step-by-Step Bit Parsing Breakdown: A rigorous bit-by-bit analysis of the top.v FSM state machine explains how the sequence 0101010101... triggered this transaction:

1) **Opcode Decoding (DETECT State):**
   - **Bit 1**: 0 → stored into mode[1].
   - **Bit 2**: 1 → stored into mode[0].
   - **Opcode Result**: {mode[1], MoSi} = 2'b01. This matches the WRITE_ADDR state opcode.
2) **Address Decoding (WRITE_ADDR State):** The FSM transitions to WRITE_ADDR and shifts in 4 address bits over the next 4 clock cycles:
   - **Bit 3**: 0 → addr = 4'b0000
   - **Bit 4**: 1 → addr = 4'b0001
   - **Bit 5**: 0 → addr = 4'b0010
   - **Bit 6**: 1 → addr = 4'b0101 (4'd5)

   Address Result: The address is decoded as 4'd5 (Sensor 4 / 5th ROM entry).
3) **ROM Metadata Lookup:** With addr = 4'd5, slave_info_rom evaluates the lookup table:

```
4'd5: begin
    in_width  = 4'd4;   out_width = 4'd4;
    in_start  = 4'd12;  out_start = 4'd12;
end
```

   The ROM dictates that Sensor 4 accepts a **4-bit payload** (in_width = 4) to be written starting at **bit index 12** (in_start = 12).
4) **Data Payload Writing (WRITE_DATA State):** The FSM transitions to WRITE_DATA and shifts in the next 4 bits of the pattern, placing each bit into write_reg[(rom_in_start + bit_count) & 4'b1111]:
   - **Bit 7**: 0 → written to write_reg[12 + 0] (bit 12 = 0)
   - **Bit 8**: 1 → written to write_reg[12 + 1] (bit 13 = 1)
   - **Bit 9**: 0 → written to write_reg[12 + 2] (bit 14 = 0)
   - **Bit 10**: 1 → written to write_reg[12 + 3] (bit 15 = 1)

   Payload Result: Bits [15:12] of write_reg are set to 4'b1010 (Hexadecimal 0xA).
5) **Register File State Modification:** Given an initial register state of 16'h0000, updating bits [15:12] to 4'b1010 yields:

$$\begin{aligned} \texttt{write_reg} &= \texttt{16'b1010_0000_0000_0000} \\ &= \texttt{16'hA000} \end{aligned} \tag{2}$$

This analysis proves that the raw sequence 0101010101... aligned perfectly with the protocol structure—providing the WRITE opcode (01), target address 4'd5 (0101), and payload 4'b1010 (0101 placed in bits [15:12]), causing the 7-segment display to change from 0000 to A000.

C. Functional Verification and Test Scenarios

Verification involved executing five distinct test cases, summarized in Table VI.

D. Quantitative Performance Comparison

Table VII provides a comparison of both methods across multiple metrics.

The latency calculations show that Method 2 provides significant performance benefits:

$$\begin{aligned} \text{Latency}_{\text{Method 1}} &= T_{\text{detect}} + T_{\text{addr}} + T_{\text{len}} + T_{\text{payload}} \\ &= 2 + 5 + 4 + N = 11 + N \text{ cycles} \end{aligned} \tag{3}$$

$$\begin{aligned} \text{Latency}_{\text{Method 2}} &= T_{\text{detect}} + T_{\text{addr}} + T_{\text{payload}} \\ &= 2 + 4 + N = 6 + N \text{ cycles} \end{aligned} \tag{4}$$

For a typical 4-bit sensor write ($N = 4$):

- Method 1 requires 15 cycles (1.50 μs at 10 MHz).
- Method 2 requires 10 cycles (1.00 μs at 10 MHz).

This represents a **33.3% reduction in transaction latency**.

TABLE VI
SYSTEM TEST SCENARIOS AND VERIFICATION MATRIX

Test ID	Scenario	Input Vectors	Expected Output	Observed Status	Method
TS-001	Read Operation (Sensor 1)	Mode: 00, Addr: 0x01	Read value matching DIP switches	PASS	1, 2
TS-002	Write Operation (Sensor 0)	Mode: 01, Addr: 0x00, Data: 0x07	Register updated to 0x07	PASS	1, 2
TS-003	Write Length Boundary Limit	Mode: 01, Addr: 0x00, Len: 15	Registers updated up to 16 bits	PASS	1
TS-004	Soft Reset Password Match	Mode: 10, Key: 1010	Registers cleared to default (0xFFFF)	PASS	1, 2
TS-005	Tri-state Isolation Test	CS Line deasserted (1)	MISO enters high-impedance state	PASS	1, 2

TABLE VII
DETAILED COMPARISON OF IMPLEMENTED SPI METHODS

Comparison Metric	Method 1: Address + Bit-Length	Method 2: ROM Lookup Table
Transaction Packet Size (Write 8b)	19 bits (2b Mode + 5b Addr + 4b Len + 8b Data)	14 bits (2b Mode + 4b Addr + 8b Data)
Control Overhead per Write	11 bits (57.8% of packet)	6 bits (42.8% of packet)
Latency (Write 4b at 10 MHz)	1.50 μs (15 clock cycles)	1.00 μs (10 clock cycles)
FPGA Logic Utilization (LUT/FF)	High (78 LUTs, 68 FFs)	Low (62 LUTs, 54 FFs)
On-Chip Memory (ROM) Usage	Zero	80 bits (5 configurations $\times$ 16 bits)
Sensor Expansion Flexibility	Dynamic (configured on the fly by software)	Static (requires hardware/HDL redesign to update ROM)
System Throughput (Continuous)	Moderate (5.26 Mbps at 10 MHz)	High (7.14 Mbps at 10 MHz)
Power Consumption (at 10 MHz)	84.0 mW	83.6 mW
Suitable Applications	Software prototyping, variable-length sensors	Production systems, fixed-spec sensor networks

X. DISCUSSION, LIMITATIONS, AND FUTURE WORK

The experimental results demonstrate that both implementations successfully address the $O(N)$ wiring complexity of traditional SPI, reducing the physical connection between the master and peripheral network to a 4-wire bus.

A. Method Selection Guidelines

Based on our comparative evaluation, we suggest the following selection guidelines:

- **Select Method 1** when working with dynamically reconfigurable sensors or during the initial prototyping phases, as it allows transaction widths to be adjusted via software without rebuilding the FPGA bitstream.
- **Select Method 2** for production systems with fixed hardware configurations. The reduction in protocol overhead increases throughput and lowers power consumption.

B. Limitations

The current design has two main limitations:

- **Static ROM Configuration:** In Method 2, changing a sensor's resolution requires updating the HDL code and re-synthesizing the FPGA bitstream.
- **Lack of Multi-Master Support:** The bus controller assumes a single SPI master. Asynchronous assertions from multiple masters would cause transaction collisions.

C. Future Work

Future iterations will explore several extensions to improve performance:

- **Dynamic RAM/Register Map Configuration:** Replacing the static ROM with a write-programmable register map. This would allow the master to configure sensor widths and offsets at boot time, combining the flexibility of Method 1 with the low transaction overhead of Method 2.
- **DMA Support and Interrupt Handlers:** Adding Direct Memory Access (DMA) engines and interrupt lines to let the controller upload sensor readings to memory without constant MCU polling.
- **Standard Interconnect Interfaces:** Wrapping the bus controller with industry-standard bus interfaces like AMBA AXI4-Lite or Wishbone, enabling integration into larger System-on-Chip (SoC) architectures.

XI. CONCLUSION

This paper presented the design, implementation, and verification of a Scalable SPI Bus Interface on a Xilinx Artix-7 FPGA. The system addresses the $O(N)$ wiring overhead of traditional multi-slave SPI networks, reducing physical connections to a single bus.

We implemented and compared two communication protocols. Method 1 (Address + Bit-Length) provides software-defined flexibility by defining data widths per transaction. Method 2 (ROM Lookup Table) stores sensor layouts on-chip, reducing latency by 33.3% and logic cell consumption by 20.5%. The integration of an ESP32 web server successfully validated the system in a hardware-in-the-loop configuration. The design provides a scalable solution for high-density, heterogeneous sensor networks in embedded and IoT applications.

APPENDIX A
SYSTEM ARCHITECTURE TABLES

This appendix provides the registers, pins, signals, and acronyms used in the design.

A. FSM State Table

Table VIII outlines the FSM state actions for `top.v`.

TABLE VIII
FSM STATE DESCRIPTIONS IN `TOP.V`

State Name	Code	Description
DETECT	3'd0	Samples 2-bit mode opcode (00: Read, 01: Write, 10: Reset, 11: WHOAMI).
READ_ADDR	3'd1	Shifts in 4-bit register address for read operations.
SEND_DATA	3'd2	Shifts out data from read_reg to MISO.
WRITE_ADDR	3'd3	Shifts in 4-bit register address for write operations.
WRITE_DATA	3'd4	Shifts in payload data from MOSI to write_reg.
RESET_PASSWORD	3'd5	Shifts in 4-bit password (4'b1111) to clear registers to 0x0000.
WHO_AM_I_STATE	3'd6	Shifts out the 8-bit configurable SENSOR_ID over MISO.

B. Signal Descriptions

Table IX describes the pins and ports of the SPI Bus Controller module.

TABLE IX
SPI BUS CONTROLLER PORT SIGNAL DESCRIPTIONS

Signal Port	Direction	Width	Description
Sclk	Input	1 bit	Serial Clock from the SPI Master.
MoSi	Input	1 bit	Master Out Slave In serial data line.
MiSo	Output	1 bit	Master In Slave Out tri-state data line.
cs	Input	1 bit	Chip Select control (active-low).
rst	Input	1 bit	Global asynchronous system reset.
sensor_data	Input	16 bits	Parallel real-time data from external sensors.

C. Acronyms and Abbreviations

Table X lists the acronyms used throughout this paper.

TABLE X
ACRONYMS AND ABBREVIATIONS

Acronym	Definition
SPI	Serial Peripheral Interface
FPGA	Field Programmable Gate Array
HDL	Hardware Description Language
FSM	Finite State Machine
ROM	Read-Only Memory
CDC	Clock Domain Crossing
MCU	Microcontroller Unit
MOSI	Master Out Slave In
MISO	Master In Slave Out
CS	Chip Select
GPIO	General Purpose Input/Output
LUT	Look-Up Table
FF	Flip-Flop
DMA	Direct Memory Access

D. Diagram Appendix and Image Reference Summary

This table summarizes the diagrams and placeholders generated in the report to guide image replacements:

- **Fig. 1:** System block diagram showing the interface architecture between the SPI master, FPGA submodules, and sensors (TikZ code included).
- **Fig. 2:** State transition diagram for the Address + Bit-Length method (TikZ code included).
- **Fig. 3:** SPI bus timing diagram for Method 1 write operations (TikZ code included).
- **Fig. 4:** State transition diagram for the ROM Lookup Table method (TikZ code included).
- **Fig. 5:** SPI bus timing diagram for Method 2 write operations (TikZ code included).
- **Fig. 7:** FPGA development board IO mapping layout diagram (TikZ code included).
- **Fig. 8:** System integration flow block diagram (TikZ code included).
- **Fig. 9:** Web Analyzer Dashboard layout screenshot showing real-time FPGA status, Canvas logic analyzer, and presets.
- **Fig. 15:** Web Analyzer simulation screenshot for Sensor Read (0x2800).
- **Fig. 16:** Web Analyzer simulation screenshot for Register Write (0x0028).
- **Fig. 17:** Web Analyzer simulation screenshot for alternating sequence Write (0xA000).
- **Fig. 13:** Oscilloscope captures of MOSI Read command and MISO Read response signals.
- **Fig. 14:** Oscilloscope captures of Write transaction signals.

REFERENCES

[1] Motorola Inc., "SPI Block Guide V03.06," Technical Report, 2003.
[2] J. H. Jenkins, *Designing with FPGAs and CPLDs*, Prentice Hall, 1994.
[3] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, McGraw-Hill, 2013.
[4] Pong P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*, Wiley-Interscience, 2008.
[5] M. D. Ciletti, *Advanced Digital Design with the Verilog HDL*, Prentice Hall, 2010.
[6] R. Keim, "Introduction to the Serial Peripheral Interface (SPI) Protocol," All About Circuits, 2016.
[7] Xilinx Inc., "Artix-7 FPGAs Data Sheet: DC and AC Switching Characteristics," DS181, 2020.
[8] Espressif Systems, "ESP32 Technical Reference Manual," v4.4, 2022.
[9] W. Valenzuela and G. Espinosa, "An FPGA-based configurable SPI master/slave controller," in *Proc. IEEE Int. Conf. Reconfigurable Comput. FPGAs*, 2011, pp. 245-250.
[10] C. Maxfield, *FPGAs: Instant Access*, Newnes, 2008.
[11] A. B. Kahng, "Power analysis and minimization in FPGA systems," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 31, no. 5, pp. 678–691, May 2012.
[12] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, 2006.
[13] W. Wolf, *FPGA-Based System Design*, Prentice Hall, 2004.
[14] S. Kilts, *Advanced FPGA Design: Architecture, Implementation, and Optimization*, IEEE Press, 2007.
[15] C. E. Cummings, "Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog," in *Proc. SNUG*, 2008.
[16] J. Gaisler, "A Gaisler Research Leon processor based system-on-chip," in *Proc. Design, Automation and Test in Europe*, 2002, pp. 120–125.
[17] Texas Instruments, "Understanding the SPI Interface," Application Report SLAA440, 2010.
[18] Analog Devices, "Serial Peripheral Interface (SPI) Interface," Reference Guide UG-1256, 2018.
[19] G. De Micheli, *Synthesis and Optimization of Digital Circuits*, McGraw-Hill, 1994.
[20] Ashenden, P. J., *The Student's Guide to VHDL*, Morgan Kaufmann, 2008.